Projet MLSF – Partie I
Un solveur SAT pour la résolution du Meowdoku

Pedro DE BEM & Amanda FONTENELE

25 octobre 2026

Résumé

Nous modélisons le jeu Meowdoku en logique propositionnelle (Question 1), implémentons l'algorithme DPLL et un lecteur DIMACS (Question 2), résolvons des grilles de bout en bout (Question 3), implémentons sept heuristiques de branchement (Question 4), les comparons sur 48 instances (Question 5) et fournissons l'interface `sat.py` demandée (Question 6). Aucune réponse fausse n'a été observée ; sur les 41 instances publiques, le mode `optimized` en résout 40 en 60 s (`public-03` n'est résolue par aucune de nos versions). Le mode `optimized` n'est pas une implémentation plus rapide : il change seulement l'heuristique (Section 6). Toutes les valeurs chiffrées proviennent des scripts `Q1_script.py` à `Q6_script.py`, dont les sorties sont dans `results/`.

Contenu du rendu et points d'attention

Fichier	Rôle
`meowdoku_to_dimacs.py`	Q1 : lecture d'une grille, encodage CNF, écriture DIMACS.
`generate_grid.py`	Q1 : générateur de grilles carrées avec solution plantée.
`dimacs_to_pylist.py`	Q2 : lecteur DIMACS vers listes Python.
`sat.py`	Q2, Q4, Q6 : DPLL (algorithmes 1 et 2), DPLL$_H$ (algorithme 3) avec compteurs, interface en ligne de commande.
`heuristics.py`	Q4 : heuristiques H1, H3, H5, H7 (versions statique et dynamique).
`solve_meowdoku.py`	Q3 : grille → DIMACS → DPLL → fichier solution, avec vérification.
`benchmark.py`	Q5, Q6 : mesures (une exécution = un processus, limite de 60 s).
`Q[1-6]_script.py`	Un script par question, qui produit les chiffres du rapport dans `results/`.
`make_figures.py`	Trace la figure 2 à partir de `results/Q5_runs.json`.
`image_grid.txt`	Grille de la figure 1 au format de l'énoncé.
`meowdoku-instances/`	18 formules Meowdoku (9 SAT, 9 UNSAT) et `expected.json`.
`public-instances/`	Instances des enseignants (41 fichiers) et `expected.json`.
`results/`	Sorties brutes des scripts (JSON et texte), dont les fichiers solution de la Q3.
`rapport/`	Sources LaTeX de ce rapport.

Table 1 – Fichiers du rendu. Le fichier `README.md` donne les commandes pour tout reproduire.

Écarts et choix par rapport à l'énoncé.

- *Q1* : le générateur ne traite que le cas $N = M = K$, ce que l'énoncé autorise. L'encodeur, lui, accepte $N \neq M$ et K quelconque.
- *Q4* : nous implémentons H1, H3, H5 et H7 (H0 sert de référence), soit 7 configurations et H0. Nous n'avons pas implémenté H2, H4, H6, ni de heuristique propre.
- *Q5* : nous mesurons le nombre de décisions, de retours arrière, le temps CPU et la part du temps passée dans l'heuristique. Nous *ne comptons pas* les propagations unitaires ni les littéraux purs (critères 3 et 4 de l'énoncé). Chaque couple (instance, heuristique) n'est exécuté *qu'une fois*.

— *Q6* : le mode `optimized` utilise l'heuristique H5d avec la même propagation que le mode `dpll`. Nous n'avons pas optimisé les structures de données (voir Section 6).

Éléments originaux. Le générateur de grilles à solution plantée (Section 1) et la construction de grilles Meowdoku *insatisfiables* à partir de ce générateur (Section 5) ne sont pas décrits dans l'énoncé. La vérification de la solution de la Q3 est indépendante du CNF. Le tracé de profils cumulés (figure 2) est emprunté à la pratique des compétitions SAT.

Conventions. Les variables sont numérotées comme en DIMACS ; un littéral l vaut p ou $\neg p$, et $\bar{l}$ désigne le littéral opposé. Toutes les mesures ont été faites sur la machine de l'annexe A, qui n'est *pas* celle de la compétition ; les temps absolus ne sont donc pas comparables à ceux de la compétition.

1 Question 1 : modélisation et génération de grilles

1.1 Variables et clauses

Pour une grille de N lignes, M colonnes et K régions $R_1, \dots, R_K$, on introduit une variable $x_{i,j}$ par case, vraie si et seulement si la case (i, j) contient un chat. Elle est numérotée $\mathrm{id}(i, j) = (i-1)M + j$, de 1 à NM. La figure 1 sert d'exemple fil rouge ($N = M = K = 6$). Les clauses sont les suivantes.

$$F_{\text{lignes}} = \bigwedge_{i,\, j<k} (\neg x_{i,j} \vee \neg x_{i,k}) \qquad \text{au plus un chat par ligne,}$$

$$F_{\text{colonnes}} = \bigwedge_{j,\, i<k} (\neg x_{i,j} \vee \neg x_{k,j}) \qquad \text{au plus un chat par colonne,}$$

$$F_{\text{régions}} = \bigwedge_{r=1}^{K} \Big[\big(\bigvee_{u \in R_r} x_u\big) \wedge \bigwedge_{\{u,v\} \subseteq R_r,\, u \neq v} (\neg x_u \vee \neg x_v) \Big] \qquad \text{exactement un chat par région,}$$

$$F_{\text{diag.}} = \bigwedge_{i<N,\, j<M} (\neg x_{i,j} \vee \neg x_{i+1,j+1}) \wedge (\neg x_{i,j+1} \vee \neg x_{i+1,j}) \qquad \text{pas de chats diagonaux.}$$

Le voisinage à huit cases est couvert sans clause supplémentaire : deux cases voisines horizontalement ou verticalement sont dans la même ligne ou la même colonne, donc déjà exclues par F_{lignes} et F_{colonnes} ; les voisines en diagonale sont les deux diagonales d'un bloc 2×2, traitées par $F_{\text{diag.}}$.

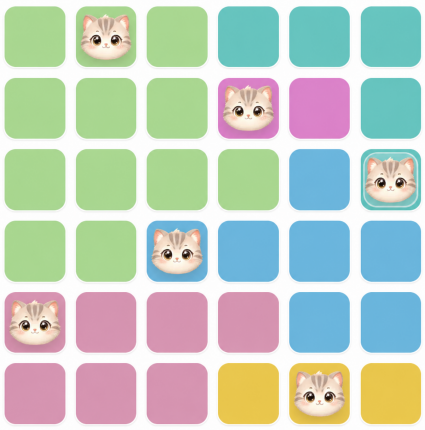


FIGURE 1 – Grille 6×6 utilisée comme exemple ; les chats montrent l'unique solution.

Correction de l'encodage. Une valuation satisfaisant $F = F_{\text{lignes}} \wedge F_{\text{colonnes}} \wedge F_{\text{régions}} \wedge F_{\text{diag.}}$ place les chats sur les variables vraies ; chaque ligne et colonne en contient au plus un, chaque région exactement un, et deux chats ne se touchent pas. Réciproquement, tout placement légal définit une

valuation qui satisfait chaque clause. Il y a donc une bijection entre les placements légaux et les modèles de F (les variables sont uniquement les cases, aucune variable auxiliaire). Sur la grille de la figure 1, nous avons testé les $6! = 720$ placements à un chat par ligne et colonne : un seul satisfait les 358 clauses, celui de la figure.

Taille (analyse de complexité). Le nombre de variables est NM. Le nombre de clauses est $N\binom{M}{2} + M\binom{N}{2} + \sum_r (1 + \binom{s_r}{2}) + 2(N-1)(M-1)$, où $s_r = |R_r|$: il est quadratique en la taille des lignes, colonnes et régions, soit $O(NM^2 + MN^2 + \sum_r s_r^2)$. Pour la figure 1, on obtient $90+90+128+50 = 358$ clauses (tailles des régions $12, 5, 2, 7, 7, 3$) ; pour une grille générée 50×50, 196 650 clauses. Nous n'éliminons pas les doublons (par exemple $\{\neg x_2, \neg x_3\}$ est émise par la règle des lignes et par celle de la région A) ; ils sont sans effet sur la satisfiabilité.

Le convertisseur `meowdoku_to_dimacs.py` lit le format de la figure 1b de l'énoncé (lignes `c` ignorées, `GRID`, `ZONES`, puis N lignes d'étiquettes), vérifie la cohérence (noms distincts, nombre de lignes et de colonnes, étiquettes utilisées exactement celles déclarées), puis écrit le fichier DIMACS dont l'en-tête est `p cnf` NM $|F|$.

1.2 Génération de grilles satisfiables

Le générateur (`generate_grid.py`, cas $N = M = K = n$) *plante* d'abord une solution, puis dessine les régions autour : la grille est satisfiable par construction.

1. *Placement des chats.* On cherche, par retour arrière avec ordre aléatoire, des colonnes $c_1, \ldots, c_n$ deux à deux distinctes avec $|c_i - c_{i+1}| \geq 2$. Il y a donc un chat par ligne et par colonne, et deux chats de lignes consécutives ne se touchent pas (ceux de lignes plus éloignées non plus). Pour $n \geq 4$, une solution existe toujours (colonnes paires croissantes puis impaires croissantes) ; pour $n = 2$ et 3, il n'y en a pas, et le générateur le signale.
2. *Croissance des régions.* La région R_i est amorcée par le chat de la ligne i. Un parcours en largeur multi-sources (file FIFO, voisins en ordre aléatoire) étend chaque région aux cases voisines encore sans région. Chaque case reçoit exactement une région, la région R_i contient son seul chat, et elle est connexe. La grille est donc valide et le placement planté en est une solution.

La graine du générateur aléatoire rend les grilles reproductibles. Le générateur garantit *au moins* une solution, pas l'unicité (la grille 5×5 de graine 1 en a 7), et ne tire pas uniformément parmi les grilles valides : le parcours en largeur produit des régions compactes de tailles inégales (de 14 à 93 cases pour $n = 50$). Le générateur produit une grille 50×50 en 4 ms ; l'essentiel du coût est celui de l'encodage ensuite.

2 Question 2 : lecture de DIMACS et algorithme DPLL

2.1 Représentation et lecture

Une formule est une liste de clauses, une clause est une liste d'entiers non nuls (convention DIMACS). La formule sans clause est `[]` (satisfiable) et celle qui contient la clause vide est par exemple `[[1, 2], [], [-1]]` (insatisfiable) ; le lecteur distingue bien les deux cas, présents dans les instances `edge_no_clause_sat` et `edge_empty_clause_unsat`. Le lecteur (`read_dimacs`) accepte les clauses sur plusieurs lignes, ignore les commentaires, et rejette avec un message explicite : en-tête absent ou mal formé, littéral hors de $1..V$, dernière clause sans `0`, nombre de clauses différent de celui annoncé. Son coût est linéaire en la taille du fichier.

2.2 DPLL : une procédure pour les trois algorithmes

Les algorithmes 1 (statut), 2 (statut et valuation) et 3 (heuristique H) de l'énoncé ne diffèrent que par le retour de la fonction et le choix de la ligne 17. Nous les implémentons par une seule procédure (`dpll_h` dans `sat.py`) ; `dpll_status` et `dpll_model` sont les algorithmes 1 et 2 avec le choix aléatoire H_0.

Procédure 1 : DPLL$_H(F)$: statut et valuation

```
1  si F contient une clause vide alors retourner (UNSAT, ∅)
2  si F ne contient aucune clause alors retourner (SAT, ∅)
3  si F contient une clause unitaire {l}, ou à défaut un littéral pur l, alors
4      (s, v) ← DPLL_H(Simplifier(F, l))
5      si s = SAT alors retourner (SAT, v ∪ {l}) sinon retourner (UNSAT, ∅)
6  l ← H(F)                          (littéral choisi : l = p signifie p ← 1, l = ¬p signifie p ← 0)
7  pour l' dans (l, l̄)
8      (s, v) ← DPLL_H(Simplifier(F, l'))
9      si s = SAT alors retourner (SAT, v ∪ {l'})
10 retourner (UNSAT, ∅)
```

Simplifier(F, l) est la formule $F[l \leftarrow 1]$: on retire les clauses contenant l et on retire $\bar{l}$ des autres. Choix d'implémentation :

- La valuation est un dictionnaire variable → booléen, construit au retour de la récursion ; UNSAT est représenté par `None`. Les variables sans valeur peuvent prendre n'importe quelle valeur ; l'interface leur donne la valeur fausse.
- La clause unitaire utilisée est la première de la liste ; à défaut, le littéral pur de plus petit numéro de variable. `simplify` construit une nouvelle liste, ce qui rend le retour arrière trivial (la formule d'origine est intacte) mais coûte une copie à chaque affectation.
- Chaque appel affecte au moins une variable : la profondeur peut atteindre le nombre de variables. Nous relevons la limite de récursion de Python et exécutons la recherche dans un thread à pile de 256 Mio (`run_deep`). Sans cela, `public-06` (1363 variables) échoue par `RecursionError`.

Complexité. Chaque nœud de l'arbre coûte $O(|F|)$: test de clause vide, recherche d'une clause unitaire, collecte des littéraux pour le littéral pur, et copie par `simplify`. La profondeur est au plus le nombre n de variables et chaque décision a deux branches, donc le pire cas est $O(2^n \cdot |F|)$: la complexité est exponentielle, comme l'énoncé le rappelle. En pratique, la taille de l'arbre dépend fortement de l'ordre de branchement (Q4–Q5).

2.3 Validation et performances

Protocole. Les 41 fichiers des enseignants sont lus, puis chacun est résolu *trois fois* par chaque version (`dpll_status` et `dpll_model`, via `sat.py`), avec 60 s par exécution ; trois fois parce que le choix de branche est aléatoire. Le statut est comparé à `expected.json` et chaque modèle est vérifié contre toutes les clauses. Les temps sont des médianes, mesurées avec huit exécutions en parallèle, donc indicatives.

Résultats. Sur 246 exécutions, aucune réponse n'est fausse. Trente-sept instances sont résolues à chaque exécution par les deux versions ; `public-03` et `public-27` ne sont jamais résolues, `public-06` et `public-12` seulement parfois. Les 19 autres instances de benchmark ont une médiane d'au plus 0,5 s. La table 2 donne les instances lentes.

Instance	Variables	Clauses	Statut	`dpll_model`	`dpll_status`
`public-03`	63	168	UNSAT	0/3, timeout	0/3, timeout
`public-06`	1363	3034	SAT	0/3, timeout	1/3, 0,5 s
`public-08`	100	900	UNSAT	3/3, 1,8 s	3/3, 0,7 s
`public-11`	729	21844	SAT	3/3, 36,8 s	3/3, 13,5 s
`public-12`	522	11636	SAT	1/3, 7,1 s	1/3, 3,3 s
`public-15`	100	850	UNSAT	3/3, 1,9 s	3/3, 2,6 s
`public-18`	459	7054	SAT	3/3, 21,0 s	3/3, 10,7 s
`public-19`	729	22060	SAT	3/3, 2,1 s	3/3, 3,3 s
`public-22`	100	850	UNSAT	3/3, 3,0 s	3/3, 3,0 s
`public-27`	181	3151	SAT	0/3, timeout	0/3, timeout
`public-30`	100	850	UNSAT	3/3, 3,5 s	3/3, 1,5 s

TABLE 2 – Instances dont la médiane dépasse 1 s ou qui ne sont pas toujours résolues. « $k/3$, t » : k exécutions sur 3 terminées en 60 s, t médiane de leurs temps.

Commentaires.

— *Temps instable.* Avec le choix aléatoire, `public-12` est résolue en 3,3 s dans une exécution de `dpll_status` et dépasse 60 s dans les deux autres ; `public-18` varie de 10,1 à 56,8 s. Les écarts entre `dpll_status` et `dpll_model` (par exemple 11 s contre 21 s sur `public-18`) sont du même ordre : les deux versions font la même recherche et nos mesures ne montrent pas que l'une soit plus rapide.

— *`public-03`* est insatisfiable avec seulement 63 variables mais aucune de nos six exécutions ne la résout en 60 s : l'arbre complet doit être parcouru. Le README des enseignants prévient que certaines instances sont difficiles pour un DPLL de base.

— *Coût du pas.* La copie de la formule à chaque affectation pèse sur les grosses instances : `public-11` et `public-19` (environ 22 000 clauses) prennent de 2 à 37 s.

3 Question 3 : résolution d'une grille Meowdoku

Le programme `solve_meowdoku.py` prend un fichier grille et écrit un fichier solution. Il enchaîne les briques précédentes (Procédure 2).

Procédure 2 : Résolution d'un fichier grille

1 lire la grille (N, M, noms de régions, $\mathrm{reg}(i,j)$) ; $F \leftarrow$ clauses de la grille (Q1)
2 écrire F dans un fichier DIMACS temporaire, puis le relire avec le lecteur de la Q2
3 $(s, v) \leftarrow \mathrm{DPLL}(F)$; **si** $s =$ UNSAT **alors** signaler « pas de solution » (code retour 1, aucun fichier écrit)
4 $P \leftarrow \{(\lfloor (t-1)/M \rfloor + 1,\ ((t-1) \bmod M) + 1) \mid t \text{ vraie dans } v\}$
5 vérifier P avec Vérifier (ci-dessous) ; **si** une violation est détectée **alors** l'afficher et s'arrêter
6 écrire : copie du fichier grille ; une ligne « `région ligne colonne` » par région, dans l'ordre de `ZONES` ; la grille où `*` marque un chat et `.` une case vide

La recherche passe bien par un fichier DIMACS, comme demandé ; le fichier temporaire est supprimé ensuite. La ligne 4 inverse $\mathrm{id}(i,j) = (i-1)M + j$: sur la figure 1, la variable 35 donne la case $(6, 5)$.

Vérification indépendante. La fonction `check_solution` n'utilise que la grille et les positions des chats, jamais le CNF : pour chaque région le nombre de chats doit être 1, et pour chaque paire de chats les lignes et colonnes doivent différer et $\max(|i-i'|, |j-j'|) \geq 2$. Elle détecte donc une erreur d'encodage, de traduction ou de solveur.

Tests. (Sorties dans `results/Q3_results.txt`.)

— Sur la grille de la figure 1b de l'énoncé, le fichier écrit contient exactement la solution de sa figure 2 (`A 3 2`, `B 2 5`, `C 1 3`, `D 5 6`, `E 4 4`, `F 6 1`).

— Sur la grille de la figure 1, dont la solution est unique, le programme retrouve les chats en $(1,2),(2,4),(3,6),(4,3),(5,1),(6,5)$.

— Des placements illégaux (deux chats sur une ligne, deux chats en diagonale, un chat déplacé dans une colonne occupée) sont tous signalés ; le placement légal ne l'est pas.

— Une grille 2×2 (régions `A A` / `B B`) est déclarée sans solution.

— Des grilles générées (graines 1, 2, 3, n de 4 à 20) sont toutes résolues et validées (table 3).

n	Variables	Graine 1	Graine 2	Graine 3
4	16	0,07 s	0,07 s	0,07 s
8	64	0,09 s	0,08 s	0,08 s
12	144	0,14 s	0,10 s	0,08 s
15	225	0,16 s	1,21 s	5,07 s
20	400	2,58 s	0,41 s	7,00 s

TABLE 3 – Temps de résolution de grilles générées $n \times n$ (une exécution, DPLL à choix aléatoire, démarrage de l'interpréteur compris ; $n = 6, 10$ omis).

Jusqu'à $n = 12$, une grille se résout en environ 0,1 s ; pour $n = 15$ et 20, le temps dépend de la grille et des choix aléatoires (de 0,16 à 7 s).

4 Question 4 : heuristiques de branchement

4.1 Heuristiques implémentées

La fonction H de la procédure 1 choisit un littéral : on essaie $l \leftarrow 1$ d'abord. Avec $\mathrm{occ}_k(l, F)$ le nombre d'occurrences de l dans les clauses de taille k de F, $\mathrm{occ}(l, F) = \sum_k \mathrm{occ}_k(l, F)$ et $k_{\min}$ la taille de la plus petite clause :

Heuristique	Score d'un littéral l	Idée	Versions
H1 (PVL)	aucun : plus petite variable de F, valeur 1	coût nul, référence	H1
H3 (LP)	$\mathrm{occ}(l, F)$	littéral le plus fréquent	H3s, H3d
H5 (LFCM)	$\mathrm{occ}_{k_{\min}}(l, F)$	le plus fréquent dans les clauses les plus courtes	H5s, H5d
H7 (LJW)	$\sum_k \mathrm{occ}_k(l, F)\, 2^{-k}$	Jeroslow–Wang : les clauses courtes pèsent plus	H7s, H7d

TABLE 4 – Heuristiques implémentées. H_0 (choix aléatoire) sert de référence.

Le littéral de score maximal est choisi ; l'égalité est départagée par le plus petit numéro de variable, puis par le littéral positif. La version *dynamique* recalcule les scores sur la formule courante à chaque décision. La version *statique* les calcule une fois sur la formule initiale, classe les littéraux, puis renvoie à chaque décision le premier littéral classé dont la variable apparaît encore dans F. Cela donne les huit configurations H_0, H1, H3s, H3d, H5s, H5d, H7s, H7d.

Respect des contraintes de l'énoncé. Heuristique fondée sur un littéral : H3, H5, H7 (H1 et H_0 choisissent la variable). Heuristique utilisant les tailles de clauses : H5, H7. Heuristique dynamique : H3d, H5d, H7d.

Raisons des choix. H3 est l'heuristique de littéral la plus simple. Une clause courte devient unitaire en premier, et la propagation unitaire est le moyen le moins cher de couper la recherche ; H5 ne regarde que les clauses les plus courtes, et H7 pondère toutes les clauses par 2^{-k}, ce qui favorise les courtes sans ignorer les autres. H1 ne coûte rien à calculer et montre ce qu'une heuristique calculée doit apporter. Proposer les versions statique et dynamique de H3, H5 et H7 permet de mesurer en Q5 ce que rapporte le recalcul et ce qu'il coûte.

Coût. Une décision dynamique est en $O(|F|)$ (parcours de F pour compter les littéraux). La version statique coûte $O(|F_0| \log |F_0|)$ une fois, mais notre implémentation parcourt aussi F à chaque décision (pour savoir quelles variables restent) : son coût est donc aussi en $O(|F|)$, avec une constante plus faible.

4.2 Validation

Les huit configurations donnent la bonne réponse, et un modèle valide en cas de SAT, sur les 11 petites instances des enseignants (interface, cas limites : 88 exécutions, aucune fausse, `results/Q4_validation.txt`), ainsi que sur les 30 instances de benchmark et les 18 formules Meowdoku de la Q5 (les instances non résolues en 60 s ne donnent pas de réponse).

5 Question 5 : comparaison des heuristiques

5.1 Protocole

Instances. Les 30 instances de benchmark des enseignants (19 SAT, 11 UNSAT) et 18 formules Meowdoku. Neuf sont satisfiables : grilles de taille $n = 10, 15, 20$, graines 1, 2, 3, du générateur de la Q1. Neuf sont insatisfiables : $n = 8, 9, 10$, mêmes graines, construites ainsi (Procédure 3).

Procédure 3 : Grille Meowdoku insatisfiable de taille n et de graine s

1 générer la grille de taille n et de graine s, avec ses chats plantés
2 $E \leftarrow$ cases sans chat, dans l'ordre de lecture ; affecter à ses trois premières cases une nouvelle région X
3 **retourner** la grille, qui a $n + 1$ régions pour n lignes

Il faut un chat par région, soit $n + 1$ chats, mais au plus un par ligne : la formule est insatisfiable. Nous donnons trois cases à la région X et non une, car une région d'une seule case donne une clause unitaire et la propagation unitaire réfute alors la formule sans aucune recherche.

Mesures. Chaque couple (instance, heuristique) est exécuté une fois, dans un processus propre, limité à 60 s, dix processus à la fois. H_0 est aléatoire : son résultat est donc un seul échantillon. Critères : nombre de *décisions* (exécutions de la ligne 17 de l'algorithme 3), temps CPU, part du temps CPU passée dans l'heuristique. Le nombre de *retours arrière* (exécutions de la branche `else`) vaut de 97,9 % à 99,5 % du nombre de décisions pour toutes les heuristiques (sur les instances résolues par toutes), donc il n'apporte presque rien en plus et n'est pas détaillé. Les compteurs ne modifient pas la recherche. Aucune des 384 exécutions n'a donné de réponse fausse (statut comparé à l'attendu, modèle vérifié sur toutes les clauses).

5.2 Résultats

	Publiques SAT (19)	Publiques UNSAT (11)	Meowdoku SAT (9)	Meowdoku UNSAT (9)	Total (48)
H_0	17	10	9	9	45
H1	18	10	8	9	45
H3s	18	10	7	9	44
H3d	18	10	9	9	46
H5s	18	10	8	9	45
H5d	19	10	9	9	**47**
H7s	17	10	7	9	43
H7d	17	10	9	9	45

TABLE 5 – Instances résolues en 60 s (effectif entre parenthèses).

La table 5 compte les instances résolues. `public-03` n'est résolue par aucune heuristique. Les autres échecs : `public-12` (H_0, H1, H3s, H5s, H7s), `public-06` (H_0, H7s, H7d), `public-11` (H3d, H7d), `sat-n20-s1` (H3s, H5s, H7s) et `sat-n20-s2` (H1, H3s, H7s).

L'effort est comparé sur les instances résolues par *toutes* les heuristiques (16 publiques SAT, 10 publiques UNSAT, 7 Meowdoku SAT, 9 Meowdoku UNSAT) : table 6.

	Publiques SAT Déc.	Publiques SAT CPU	Publiques UNSAT Déc.	Publiques UNSAT CPU	Meowdoku SAT Déc.	Meowdoku SAT CPU	Meowdoku UNSAT Déc.	Meowdoku UNSAT CPU	Part heur.
H_0	13,1 k	70,0 s	9,4 k	15,3 s	3,1 k	2,1 s	24,1 k	15,3 s	1,8 %
H1	6,9 k	66,4 s	5,2 k	7,2 s	39,1 k	29,0 s	1,4 k	1,5 s	2,0 %
H3s	24,1 k	31,9 s	3,4 k	5,0 s	33,2 k	19,4 s	33,0 k	15,5 s	3,3 %
H3d	28,0 k	50,9 s	2,3 k	3,4 s	2,4 k	3,4 s	48,4 k	17,2 s	8,7 %
H5s	6,8 k	37,0 s	1,2 k	0,5 s	33,2 k	17,6 s	33,0 k	18,5 s	2,9 %
H5d	57,9 k	77,3 s	1,0 k	0,5 s	2,4 k	3,5 s	48,4 k	18,8 s	10,8 %
H7s	5,3 k	29,0 s	1,2 k	0,6 s	33,2 k	16,4 s	33,0 k	15,2 s	3,0 %
H7d	18,4 k	62,7 s	1,0 k	0,5 s	2,4 k	3,8 s	48,4 k	18,4 s	9,7 %

TABLE 6 – Total des décisions (milliers) et du temps CPU par groupe, sur les instances résolues par toutes les heuristiques, et part du temps CPU passée dans l'heuristique (sur les quatre groupes).

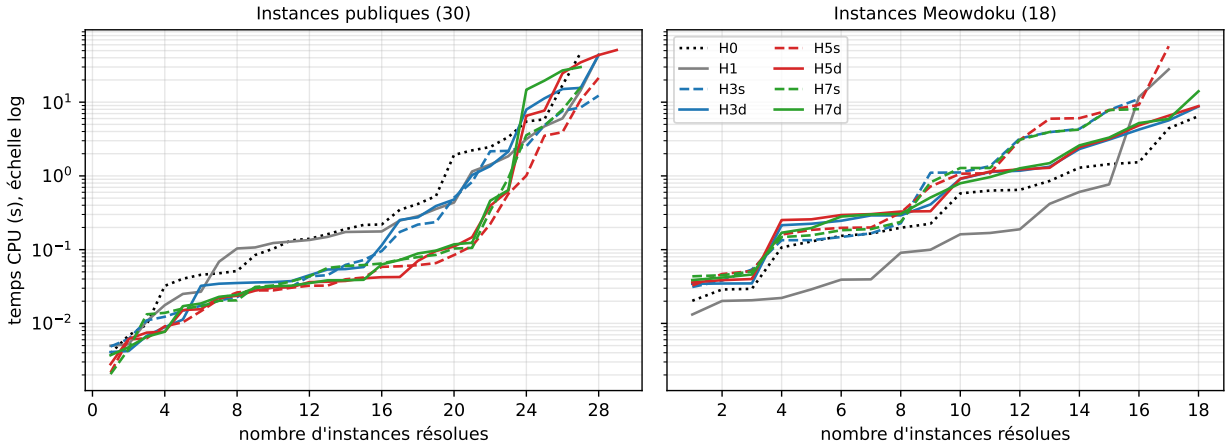


FIGURE 2 – Profils cumulés : pour chaque heuristique, temps CPU des instances résolues, triés par ordre croissant (échelle logarithmique). Plus une courbe est basse et longue, mieux c'est.

5.3 Analyse

Les résultats sont-ils conformes à nos attentes ? En partie. Nous attendions que les heuristiques battent le choix aléatoire, que le dynamique batte le statique et que les heuristiques à tailles de clauses battent les simples.

— *Heuristique contre aléatoire.* Vrai sur les instances publiques UNSAT : H5 et H7 font environ 1 000 décisions contre 9 400 pour H_0, et 0,5 à 0,6 s de CPU contre 15,3 s. Faux sur Meowdoku : en SAT, H_0 résout les sept grilles en 2,1 s, plus vite que toutes les autres ; en UNSAT, les heuristiques à fréquence font plus de décisions que H_0 (33 000 à 48 400 contre 24 100), seule H1 fait mieux (1 400).

— *Dynamique contre statique.* Vrai sur Meowdoku SAT (2 400 décisions contre 33 200). Faux sur les publiques SAT (H5d : 57 900 décisions, H5s : 6 800) et sur Meowdoku UNSAT (48 400 contre 33 000).

— *Tailles de clauses.* Sur les publiques UNSAT, H5 et H7 sont bien meilleures que H1 et H3 ; sur les publiques SAT, H7s est la plus rapide (29,0 s), mais H3d est plus rapide que H5d et H7d.

Sur les grilles Meowdoku, les trois versions dynamiques prennent *exactement* les mêmes décisions (mêmes totaux). Nous supposons que c'est dû à la structure de la formule : presque toutes les clauses sont binaires et négatives (au plus un chat), donc les trois scores désignent le même littéral. Nous n'avons pas vérifié cette explication. Sur Meowdoku UNSAT, H1 est la meilleure : nous supposons que brancher dans l'ordre des cases épuise rapidement les premières lignes et révèle le manque de chats, sans avoir vérifié non plus.

Dans quelle mesure une heuristique améliore-t-elle DPLL (algorithme 2) ? Cela dépend de la famille. Sur les publiques UNSAT, H5 et H7 divisent le temps CPU par 25 à 30. Sur les publiques SAT, H7s prend 29,0 s contre 70,0 s pour H_0. Sur Meowdoku, il n'y a pas de gain avec nos heuristiques à fréquence (sauf H1 en UNSAT). Le nombre d'instances résolues augmente peu : 47 pour H5d contre 45 pour H_0, qui échoue sur `public-06` et `public-12` ; comme H_0 est un seul tirage, ces écarts sont indicatifs.

Le recalcul dynamique est-il rentable ? Le calcul de l'heuristique représente de 9 à 11 % du temps CPU en dynamique contre environ 3 % en statique. Le recalcul rapporte sur Meowdoku SAT : de 4 à 6 fois plus rapide (3,4 à 3,8 s contre 16,4 à 19,4 s) et les neuf grilles sont résolues, ce qu'aucune version statique ne fait. Il rapporte aussi sur `public-12`, que ni H_0 ni les versions statiques ne résolvent. Il ne rapporte pas sur les publiques SAT (H3d, H5d et H7d sont plus lentes que leurs versions statiques : 50,9, 77,3 et 62,7 s contre 31,9, 37,0 et 29,0 s) ni sur les UNSAT (temps comparables). Aucune heuristique n'est la meilleure partout ; selon le nombre d'instances résolues, premier critère du classement, H5d est la meilleure.

Limites de l'étude. Une seule exécution par couple ; H_0 n'est qu'un échantillon ; les totaux ne portent que sur les instances résolues par toutes les heuristiques, ce qui exclut les instances difficiles et sous-estime les heuristiques qui résolvent plus d'instances ; dix processus tournent en parallèle, ce qui perturbe les temps ; les propagations unitaires et littéraux purs ne sont pas comptés.

6 Question 6 : interface et mode optimisé

Interface. La commande est `python sat.py instance.cnf`, avec les options `--mode dpll`, `--mode optimized` et `--output-model`. Le mode par défaut est `dpll`. La sortie standard contient `s SATISFIABLE` ou `s UNSATISFIABLE` et, avec `--output-model` pour une formule satisfiable, une ligne `v` listant les variables de 1 à V avec leur signe et terminée par `0` (variable sans valeur : fausse) ;

aucun modèle pour une formule insatisfiable. Rien d'autre n'est écrit sur la sortie standard, les erreurs vont sur la sortie d'erreur. Les deux modes partagent le même format et donnent le même statut.

Mode `optimized`. C'est DPLL$_H$ avec l'heuristique H5d. Nous l'avons choisie d'après la Q5, parce qu'elle résout le plus d'instances (47 sur 48) et que le nombre d'instances résolues est le premier critère de classement. Elle est déterministe, donc reproductible. *La propagation et les structures de données sont celles du mode `dpll`* : ce n'est pas une implémentation plus rapide, seulement un autre ordre de branchement.

Vérification. Les deux modes ont été exécutés par la ligne de commande sur les 41 instances publiques, une fois chacun, 60 s, huit à la fois. Une réponse est correcte si le statut est le bon et, pour SAT, si le modèle satisfait toutes les clauses.

Mode	Réponses correctes (sur 41)	Temps total de ces réponses
`dpll`	39	51,8 s
`optimized`	40	134,7 s

TABLE 7 – Les deux modes sur les 41 instances publiques.

Aucune réponse n'est fausse. Le mode `dpll` ne résout ni `public-03` ni `public-27`. Le mode `optimized` résout `public-27` (40,7 s) mais pas `public-03`. Sur les 39 instances résolues par les deux, il est plus lent (94,0 s contre 51,8 s), surtout sur `public-06` (35,1 s contre 0,6 s) et `public-12` (25,9 s contre 12,4 s), que le mode `dpll` a résolues vite grâce à des choix aléatoires favorables (cf. Q2).

Limites et pistes. Le gain d'une instance (`public-27`) se fait avec une marge faible : 40,7 s et 35,1 s sur `public-06` pour une limite de 60 s, sur une machine différente de celle de la compétition. Le goulot d'étranglement est la propagation : la copie de F à chaque affectation et le parcours complet pour trouver les clauses unitaires. Les pistes les plus directes sont une propagation unitaire par compteurs ou *watched literals* avec un historique d'affectations défaisable (plus de copie), des index littéral → clauses, et une boucle itérative. Nous ne les avons pas réalisées.

7 Bilan

L'encodage est vérifié de façon exhaustive sur la grille exemple et, indépendamment, par `check_solution` ; le solveur est correct sur toutes les instances disponibles (aucune réponse fausse sur les 41 instances publiques et les 18 formules Meowdoku). Ses faiblesses sont la vitesse (une instance publique non résolue, plusieurs résolues près de la limite) et le protocole de mesure (une exécution par couple, critères partiels), signalées ci-dessus. Le point le plus utile pour la suite serait de remplacer la propagation par copie par une propagation incrémentale.

A Environnement expérimental

Machine	Acer Nitro ANV16-42-R3JW
Processeur	AMD Ryzen 7 260, x86_64, 8 cœurs physiques, 16 CPU logiques
Mémoire	14 Gio de RAM (DDR5, 5600 MT/s), 4 Gio de swap
Système	Ubuntu 26.04.1 LTS, noyau Linux 7.0.0-38-generic
Python	CPython 3.14.4 (la compétition utilise Python 3.12 ; la syntaxe du code a été vérifiée pour 3.12, mais il n'a pas été exécuté sous 3.12)
Parallélisme	Q2 : 8 processus à la fois ; Q5 : 10 ; Q6 : 8 ; Q3 : un seul

TABLE 8 – Configuration utilisée pour toutes les mesures. Elle diffère de celle de la compétition (Core Ultra 7 155H, Ubuntu 22.04, Python 3.12, Docker limité à 4 CPU et 8 Gio).

B Reproduction

Depuis la racine du rendu, avec Python ≥ 3.12 (la figure demande `matplotlib`) :

```
python3 sat.py public-instances/instances/smoke_01_sat.cnf \
        --mode optimized --output-model
python3 meowdoku_to_dimacs.py image_grid.txt grid.cnf
python3 generate_grid.py 8 grille8.txt --seed 1
python3 solve_meowdoku.py image_grid.txt solution.txt
python3 Q1_script.py     # ... jusqu'à Q6_script.py ; écrit dans results/
python3 make_figures.py                            # rapport/figures/cactus.pdf
```

Les scripts Q2, Q5 et Q6 lancent des centaines d'exécutions limitées à 60 s et peuvent durer longtemps.